

Detección de intrusiones en redes

Comparación entre Random Forest y MLP sobre CICIDS-2017

Lucas Calierno

Universidad Nacional del Sur

Septiembre 2026

Métodos para el análisis de datos I

¿Por qué detectar intrusiones?

- Las redes modernas generan grandes cantidades de tráfico continuamente.
- Entre ese tráfico puede existir actividad maliciosa, como ataques, escaneos, intentos de acceso no autorizado, entre otras.
- Un Sistema de Detección de Intrusiones (IDS) busca identificar automáticamente este comportamiento en tiempo real.
- El machine-learning permite automatizar esta tarea a partir de características extraídas del mismo tráfico de red.

Objetivo

Clasificar los tipos de tráficos utilizando modelos de machine-learning.

Problema planteado

El trabajo busca comparar dos enfoques:

Machine Learning clásico

Random Forest

- Modelo basado en árboles.
- Muy utilizado en datos tabulares.
- Robusto y relativamente simple.

Deep Learning

MLP

- Red neuronal multicapa.
- Gran capacidad para aprender relaciones no lineales.
- Mayor cantidad de parámetros y ajuste.

Pregunta

¿Vale la pena un el costo de un modelo de Deep Learning para datos estructurados?

Ali et al. (2025)

Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study

- Estudia diferentes modelos clásicos y de Deep Learning para detección de intrusiones.
- Utiliza el dataset CICIDS-2017.
- entre sus conclusiones principales, se encuentra que los modelos clásicos pueden superar a los modelos profundos en este problema.

Idea del trabajo

Replicar el análisis utilizando Random Forest y MLP, y comparar los resultados obtenidos con los del paper

Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset 2017

- Dataset de tráfico de red utilizado para evaluar sistemas IDS.
- Contiene tráfico capturado durante cinco días.
- Incluye tráfico benigno y diferentes tipos de ataques.
- La versión original contiene:
 - 2.830.743 registros.
 - 79 columnas.
 - Una variable objetivo denominada 'Label'.

El dataset fue obtenido del siguiente link: <https://cicresearch.ca/CICDataset/CIC-IDS-2017/>

Preprocesamiento: Tratamiento de variables predictoras

Los datos fueron tratados previos al modelado.

- Se reportaron 308381 datos duplicados. Estos fueron eliminados para evitar malas lecturas.
- Se reportaron 353 valores faltantes. Los mismos fueron reemplazados por la respectiva media. Este método fue usado debido a que es simple y efectivo para datos numéricos. Además, se evitó eliminar posibles datos que podrían ser importantes.
- Se reportaron 3128 valores infinitos, tanto positivos como negativos. Se decidió quitar tales registros para evitar posibles errores.
- Se dividió el conjunto de datos en entrenamiento (80 %) y prueba (20 %)
- Se estandarizó el conjunto de datos de entrenamiento, y luego se aplicó la transformación al conjunto de prueba
- Se evaluó la correlación, y se encontraron 34 variables con un coeficiente de correlación mayor a 0,85. En estos casos se eliminó una de las variables, para reducir redundancia

Preprocesamiento: Tratamiento de variable objetivo

La variable objetivo ('Label') representa el tipo de tráfico. Entre las categorías encontramos la clase benign, que indica tráfico normal, y 14 tipos más de ataque.

- BENIGN
- DoS Hulk
- DDoS
- PortScan
- DoS GoldenEye
- DoS slowloris
- DoS Slowhttptest
- FTP-Patator
- SSH-Patator
- Bot
- Web Attack – Brute Force
- Web Attack – XSS
- Web Attack – SQL Injection
- Infiltration
- Heartbleed

Para simplificar la clasificación, las etiquetas fueron consolidadas en siete clases.

Preprocesamiento: Tratamiento de variable objetivo

Se utilizó el siguiente agrupamiento:

Etiquetas originales	Clase final
BENIGN	BENIGN
DoS Hulk, DoS GoldenEye, DoS slowloris, Dos Slowhttpstest	DoS
DDoS	DDoS
PortScan	PortScan
Bot	Bot
Web Attack – Brute Force, Web Attack – XSS, Web Attack – SQL Injection	Web Attack
Hearthbleed, Infiltration, FTP-Patator, SSH-Patator	Other

La idea es categorizar la variable objetivo entre los principales tipos de ataques. El agrupamiento fue seguido del paper de referencia.

Preprocesamiento: Tratamiento de variable objetivo

Otro problema con la variable objetivo, son las frecuencias:

Clase	Cantidad
BENIGN	2.095.057
Dos	193.745
DDoS	128.014
PortScan	90.694
Other	9.197
Web Attack	2.143
Bot	1.948

Problema

Existe un fuerte desbalance entre clases. La clase 'Benign' domina ampliamente.

Preprocesamiento: Tratamiento de variable objetivo

El desbalance entre clases puede provocar errores en el modelado, y una baja detección de las clases minoritarias. Para abordar el problema del desbalance entre clases, se decidió aplicar SMOTE (Synthetic Over-Sampling Technique).

Esta técnica genera nuevas observaciones sintéticas de las clases minoritarias a partir de las ya existentes. Esto permite aumentar la cantidad de observaciones, sin duplicar observaciones. El resultado es un dataset balanceado, con igual cantidad de observaciones en cada clase.

Importante

SMOTE se aplica sobre el conjunto de entrenamiento únicamente

Preprocesamiento: Pipeline de preprocesamiento

- 1 Eliminación de duplicados.
- 2 Imputación de valores faltantes por respectiva media.
- 3 Eliminación de valores infinitos.
- 4 Consolidación de etiquetas en variable objetivo.
- 5 División train/test.
- 6 Estandarización.
- 7 Eliminación de variables correlacionadas.
- 8 SMOTE sobre train.

Random Forest: Marco teórico - Árboles de decisión

Un árbol de decisión divide los datos mediante una secuencia de reglas hasta llegar a una predicción. Este realiza una serie de preguntas sobre una característica particular del dato, hasta llegar a una predicción

Ventajas

- Son fáciles de interpretar.
- Pueden capturar relaciones no lineales.
- No requieren asumir una distribución específica de los datos.

Principal desventaja

Los árboles de decisión presentan una varianza muy alta, pues pequeños cambios en los datos generan árboles muy distintos

Random Forest: Marco teórico - Idea general

Para solucionar esto, se introduce el concepto de Random Forest

¿Qué es Random Forest?

Random Forest es un algoritmo de aprendizaje supervisado que combina múltiples árboles de decisión para realizar una predicción.

- Cada árbol realiza una predicción de manera independiente.
- Los árboles se construyen utilizando diferentes muestras de los datos y subconjuntos de variables.
- Finalmente, se combinan las predicciones de todos los árboles.

La idea intuitiva es, un árbol puede equivocarse, muchos árboles a la vez es más raro. El modelo se vuelve más estable y robusto a los cambios. Para lograrlo, los árboles tienen que ser distintos entre sí.

Random Forest: ventajas y limitaciones

A continuación se presentan las principales ventajas y limitaciones de este método

Ventajas

- Robusto frente a ruido y variaciones en los datos.
- Reduce la varianza respecto de un único árbol.
- Puede capturar relaciones no lineales.
- Requiere poca preparación de los datos.
- Difícil sobreajustarlo.

Limitaciones

- Predicciones lentas en tiempo real debido a la cantidad de árboles
- Tiene un consumo de memoria elevado.
- Difícil de interpretar una predicción.
- Un bosque muy grande puede aumentar mucho el costo computacional.

Random Forest: hiperparámetros elegidos

Se eligieron los siguientes hiperparámetros:

Parámetro	Valor
n_estimators	100
random_state	42
n_jobs	1

- `n_estimators`: cantidad de árboles de decisión. Elegido como balance entre capacidad de predicción y costo computacional.
- `random_state`: Fijación de semilla aleatoria. No aporta ventaja estadística
- `n_jobs`: cantidad de procesos utilizados. Se eligió 1 empíricamente para menor costo computacional

El resto de hiperparámetros utilizados se dejaron por defecto los de la librería sklearn

¿Que es una neurona artificial?

Es una unidad de procesamiento que recibe entradas, las combina linealmente, suma un sesgo y aplica una función de activación para producir una salida.

Cada neurona realiza una transformación:

$$y = \sigma(z), \quad \text{donde} \quad z = \sum_i w_i x_i + b$$

- x_i : valores de entrada.
- w_i : pesos de cada entrada. Indican la importancia de cada una.
- b : sesgo (bias). Permite ajustar el umbral.
- σ : función de activación. Decide si la neurona se "prende." o no
- y : salida de la neurona.

¿Que es Multilayer Perceptron (MLP)?

Es una red neuronal, que organiza varias neuronas artificiales, y las apila en forma de capas

Las capas de neuronas se dividen en tres grupos:

- Capa de entrada: Primera capa de neuronas que reciben los datos
- Capas ocultas: Capas intermedias que procesan la información, y aprenden las relaciones mas complejas.
- Capa de salida: Capa final encargada de realizar la predicción.

Cada capa de neuronas aprende relaciones más complejas de los datos. Con suficientes capas, se puede aproximar cualquier función, sin importar su complejidad.

MLP: Marco teórico - Ventajas y limitaciones

A continuación, se presentan las principales ventajas y limitaciones de este método

Ventajas

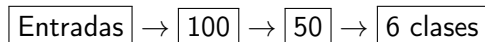
- Captura relaciones y patrones complejos entre las variables
- Permite ajustar su arquitectura como cantidad de capas y neuronas, para optimizar su rendimiento
- Es un modelo muy flexible, que se puede adaptar para muchos usos.

Limitaciones

- Requiere ajuste de muchos hiperparámetros.
- Es fácil caer en un sobreajuste si el modelo es demasiado complejo en comparación a los datos
- Entrenamiento costoso computacionalmente.
- Muy difícil de explicar las predicciones realizadas.

MLP: arquitectura utilizada

Se utilizó la siguiente arquitectura



- Capa de entrada: Recibe información del dataset.
- 2 Capas ocultas, de 100 y 50 neuronas, respectivamente. Fueron elegidas como un balance entre capturar las relaciones de las variables, manteniendo un costo computacional razonable.
- Activación: ReLU. Es la función de activación estándar, debido a su simplicidad computacional
- Optimizador: Adam. Método basado en descenso por gradiente
- Capa de salida multiclase. Es la encargada de realizar las predicciones.

MLP: hiperparámetros utilizados

Los principales hiperparámetros utilizados fueron:

Parámetro	Valor
hidden_layer_sizes	(100, 50)
activation	ReLU
solver	Adam
alpha	0.0001
batch_size	512
learning_rate_init	0.0001
max_iter	100
early_stopping	True
validation_fraction	0.1
n_iter_no_change	5

Resultados: Métricas

Los resultados generales del modelo fueron:

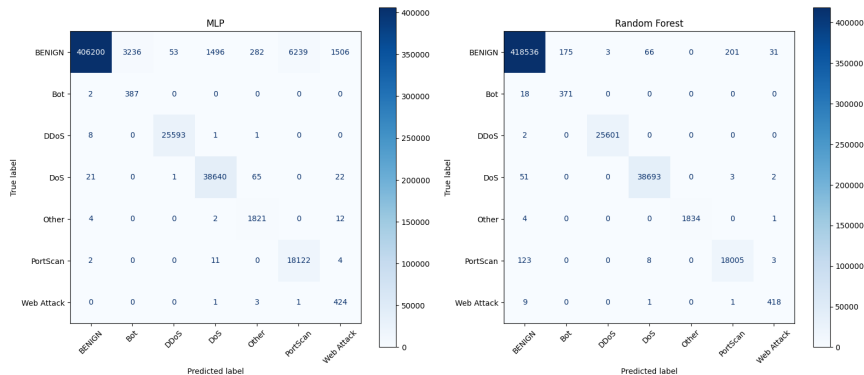
Modelo	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
MLP	0.97	0.69	0.99	0.75
Random Forest	0.99	0.94	0.98	0.96

Podemos observar que el modelo Random forest supera al MLP en Accuracy, Precision y F1-score, mientras que el modelo MLP presenta mayor Recall

Pregunta

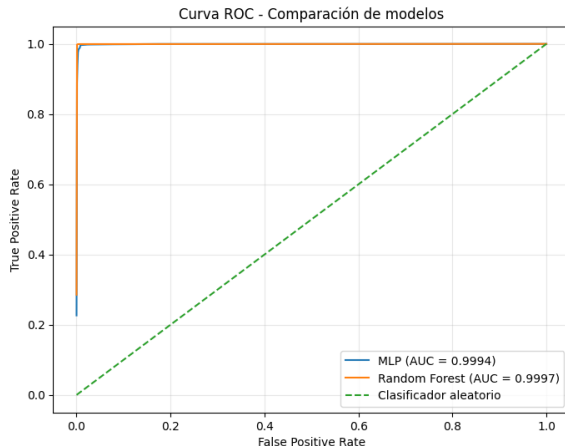
Podemos concluir que Random Forest es mejor modelo que MLP?

Resultados: Matrices de confusión



Se presentan las matrices de confusión de ambos modelos, que comparan la clase real vs la clase predicha.

Resultados: Curvas ROC



Se observa que ambos modelos presentan una curva casi perfectamente cuadrada, lo que refleja el buen desempeño de los modelos.

Resultados: propios vs paper

	RF paper	MLP paper	RF propio	MLP propio
Accuracy	99,88 %	97,00 %	99,86 %	97,42 %
F1 macro	97,46 %	82,00 %	96,08 %	75,28 %
F1 ponderado	100,00 %	98,00 %	99,86 %	97,86 %

Podemos observar que los resultados obtenidos por nuestro modelado son semejantes a los del paper de referencia.

Por que puede ganar Random Forest?

La principal ventaja del random forest reside en que el dataset es fundamentalmente tabular y con datos numéricos, en lo cual se especializa este modelo

¿Siempre es mejor el Random Forest?

Veamos el siguiente paper

Tang, W., & Fei, L. (2025). *Handwritten digit recognition based on SVM*. In **Proceedings of the 2024 2nd International Conference on Frontiers of Intelligent Manufacturing and Automation (CFIMA '24)** (pp. 324–331). Association for Computing Machinery.

Este paper compara algoritmos de machine learning clásicos con deep learning, sobre un dataset compuesto por imágenes de dígitos escritos a mano, donde cada muestra representa un número del 0 al 9.

El paper llega a que MLP obtiene mejores resultados, con accuracy de 78,3 % y F1 de 74 %, contra Random Forest, con accuracy de 64,7 % y F1 de 65,7 %

Resultados: Conclusiones

En conclusión, Random forest presenta mejor desempeño en datos tabulares. Sin embargo, podemos afirmar que es necesariamente mejor que MLP?

Como vimos antes, MLP presenta mayor recall que Random Forest. Además, si comparamos las matrices de confusión de ambos modelos, se observa que MLP captura mejor los ataques, es decir, tiene menos cantidad de ataques clasificados como 'Benign'. (37 falsos 'Benign' contra 207)

En un contexto en el que el costo de recibir un ataque sea muy alto, podría ser mejor perder accuracy y reportar falsos ataques, pero capturar todos los ataques.

Idea general

El resultado depende siempre del contexto, no es absoluto

Conclusiones finales

- CICIDS-2017 presenta un problema de clasificación tabular con un fuerte desbalance de clases.
- El preprocesamiento y SMOTE son fundamentales para obtener una comparación adecuada.
- Random Forest resulta especialmente adecuado para este tipo de datasets.
- MLP ofrece una mayor flexibilidad, pero requiere mayor cuidado en el escalado y configuración.
- Deep Learning no es necesariamente superior por ser más complejo.
- La elección debe considerar desempeño, estructura de los datos, costo computacional y costo de error.

Conclusión general

El mejor modelo depende del contexto del problema.

Referencias



Ali, M. L. et al. (2025).

Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study.
Applied Sciences, 15(4), 1903.



Hastie, T., Tibshirani, R. & Friedman, J. (2009).

The Elements of Statistical Learning.
Springer.



Goodfellow, I., Bengio, Y. & Courville, A. (2016).

Deep Learning.
MIT Press.



Tang, W. & Fei, L. (2025).

Handwritten digit recognition based on SVM.

In Proceedings of the 2024 2nd International Conference on Frontiers of Intelligent Manufacturing and Automation (CFIMA '24) (pp. 324–331). Association for Computing Machinery.